

LAPORAN PRAKTIKUM
STRUKTUR DATA
“ABSTRACT DATA TYPE”



Disusun oleh:

Ndaru Pradana Gatot Suseno
2411532007

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga Laporan Praktikum Struktur Data 1 dapat diselesaikan. Laporan ini disusun sebagai dokumentasi pelaksanaan praktikum sekaligus sarana untuk menjelaskan penerapan konsep Abstract Data Type (ADT), class, object, enkapsulasi, constructor, dan method dalam bahasa pemrograman Java.

Kegiatan praktikum berfokus pada pemodelan data waktu melalui class `Jam_2411532007` beserta dua class driver yang digunakan untuk menguji validasi waktu, konversi ke satuan detik, perbandingan, penjumlahan, perpindahan detik, dan perhitungan durasi. Adapun bagian laporan tugas praktikum membahas class `mobil_2411532007` dan `mobilDriver_2411532007` yang menerapkan pengelolaan koleksi objek mobil menggunakan array.

Penulis menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah memberikan arahan selama kegiatan berlangsung. Isi laporan ini disesuaikan dengan source code pada package `pekan1_2411532007` dan hasil eksekusi program. Penulis menyadari masih terdapat kekurangan dalam penyusunan laporan, sehingga kritik dan saran yang membangun sangat diharapkan sebagai bahan perbaikan untuk praktikum berikutnya.

Padang, Juli 2026

Ndaru Pradana Gatot Suseno

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	2
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.2 Pelaksanaan Praktikum.....	4
2.2.1 Class Jam_2411532007	4
2.2.2 Class JamDriver_2411532007.....	6
2.2.3 Class JamDriver2_2411532007.....	7
BAB III KESIMPULAN.....	9
DAFTAR PUSTAKA.....	10
LAPORAN TUGAS PRAKTIKUM.....	11
A. Class mobil_2411532007.....	11
B. Class mobilDriver_2411532007	13
C. Hasil dan Analisis Tugas.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data berkaitan dengan cara data direpresentasikan, disimpan, dan diolah agar suatu program dapat bekerja secara teratur. Sebelum mempelajari struktur data dinamis seperti linked list, stack, queue, tree, dan graph, diperlukan pemahaman mengenai pemodelan data melalui class dan object. Dalam Java, class digunakan sebagai rancangan yang memuat atribut serta method, sedangkan object merupakan instansiasi dari rancangan tersebut.

Konsep Abstract Data Type menekankan pemisahan antara data dan operasi yang dapat dilakukan terhadap data tersebut. Pengguna class tidak perlu mengetahui seluruh detail implementasi internal selama tersedia antarmuka operasi yang jelas. Enkapsulasi mendukung prinsip ini dengan membatasi akses langsung terhadap atribut dan menyediakan method untuk membaca atau mengubah nilai secara terkendali.

Pada Praktikum 1, konsep tersebut diterapkan melalui pemodelan objek jam. Class `Jam_2411532007` menyimpan jam, menit, dan detik serta menyediakan operasi validasi, konversi, perbandingan, penjumlahan, dan perpindahan waktu. Selanjutnya, tugas praktikum menerapkan array of object untuk mengelola koleksi mobil melalui operasi tambah, tampil, ubah harga, dan hapus.

1.2 Tujuan Praktikum

1. Memahami konsep Abstract Data Type, class, object, atribut, dan method pada Java.
2. Menerapkan enkapsulasi melalui atribut private, getter, setter, dan operasi yang disediakan class.
3. Menguji operasi pada class `Jam_2411532007` menggunakan `JamDriver_2411532007` dan `JamDriver2_2411532007`.
4. Menganalisis hubungan antara nilai input, proses method, dan keluaran program.
5. Menerapkan array of object untuk mengelola koleksi mobil pada tugas praktikum.

1.3 Manfaat Praktikum

Praktikum ini memberikan dasar untuk menyusun program yang modular, mudah diuji, dan lebih terorganisasi. Pemahaman terhadap hubungan antara class utama dan class driver membantu praktikan memisahkan tanggung jawab penyimpanan data dari proses pengujian. Selain itu, penerapan array of object menjadi pengantar untuk memahami struktur data yang menyimpan sekumpulan objek secara teratur.

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Abstract Data Type

Abstract Data Type (ADT) merupakan model data yang didefinisikan berdasarkan kumpulan nilai dan operasi yang tersedia. Fokus ADT terletak pada apa yang dapat dilakukan terhadap data, bukan pada detail bagaimana data tersebut disimpan. Dalam Java, ADT dapat diwujudkan melalui class dengan atribut sebagai representasi keadaan dan method sebagai kumpulan operasi.

2.1.2 Class dan Object

Class adalah rancangan yang mendefinisikan karakteristik suatu objek. Atribut menyimpan keadaan objek, sedangkan method menentukan perilaku atau operasi yang dapat dilakukan. Object dibuat menggunakan constructor dan setiap object mempunyai nilai atributnya sendiri. Pada praktikum ini, `Jam_2411532007` menjadi class utama, sedangkan object `a`, `b`, `j1`, dan `j2` dibuat pada class driver.

2.1.3 Enkapsulasi

Enkapsulasi dilakukan dengan menempatkan atribut pada akses `private` sehingga atribut tidak diubah secara langsung dari luar class. Nilai atribut dapat diakses melalui `getter` dan diubah melalui `setter` atau method tertentu. Pendekatan ini membantu menjaga konsistensi keadaan object dan mengurangi ketergantungan antarclass.

2.1.4 Constructor dan Method

Constructor dipanggil ketika object dibuat dan digunakan untuk memberikan nilai awal pada atribut. Method dapat berupa method instance yang bekerja terhadap suatu object, seperti `plus()` dan `nextSeconds()`, maupun method static yang dapat dipanggil melalui nama class, seperti `isValid()`, `fromSeconds()`, dan `durasiDetik()`.

2.1.5 Array of Object

Array of object merupakan array yang setiap elemennya menyimpan referensi ke sebuah object. Pada class `mobil_2411532007`, array `koleksiMobil` digunakan untuk menampung maksimal sepuluh object mobil. Variabel

jumlahMobil mencatat banyaknya elemen yang telah terisi sehingga proses penambahan, pencarian, perubahan, dan penghapusan dapat dilakukan secara terkontrol.

2.2 Pelaksanaan Praktikum

2.2.1 Class Jam_2411532007

Class Jam_2411532007 menyimpan tiga atribut integer, yaitu hh, mm, dan ss. Atribut tersebut mewakili jam, menit, dan detik. Class menyediakan validasi nilai, constructor, getter, setter, konversi ke detik, pembentukan object dari total detik, perbandingan, operasi aritmetika waktu, perpindahan detik, perhitungan durasi, dan representasi string.

```

1 package pekan1_2411532007;
2
3 public final class Jam_2411532007 {
4     private int hh;
5     private int mm;
6     private int ss;
7
8     public static boolean isValid(int h, int m, int s) {
9         return ( 0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
10    }
11
12    public Jam_2411532007(int h, int m, int s) {
13        this.hh = h ; this.mm = m ; this.ss = s;
14    }
15
16    public int getHour() {return hh;}
17    public int getMinute() {return mm;}
18    public int getSecond() {return ss;}
19
20    public void setHour(int h) {this.hh = h;}
21    public void setMinutes(int m) {this.mm = m;}
22    public void setSecond(int s) {this.ss = s;}
23
24    public int toSeconds () {return hh * 3600 + mm * 60 + ss;}
25
26    public static Jam_2411532007 fromSeconds (int total) {
27        if (total < 0 ) throw new IllegalArgumentException("detik negatif");
28        total %= 24 * 3600;
29        int h = total /3600;
30        int m = total /60;
31        int s = total % 60;
32        return new Jam_2411532007(h,m,s);
33    }
34
35    public int compareTo(Jam_2411532007 other) {
36        return Integer.compare(this.toSeconds(),other.toSeconds());
37    }
38
39    public boolean equals(Object o) {
40        if (! (o instanceof Jam_2411532007 j)) return false;
41        return hh == j.hh && mm == j.mm && ss == j.ss;
42    }
43
44    public boolean equals(Object o) {
45        if (! (o instanceof Jam_2411532007 j)) return false;
46        return hh == j.hh && mm == j.mm && ss == j.ss;
47    }
48
49    public int hashCode() {
50        return java.util.Objects.hash(hh, mm, ss);
51    }
52
53    public Jam_2411532007 plus(Jam_2411532007 other) {
54        return fromSeconds(this.toSeconds() + other.toSeconds());
55    }
56
57    public Jam_2411532007 minus(Jam_2411532007 other) {
58        return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24 *3600 ));
59    }
60
61    public Jam_2411532007 nextSeconds() {
62        return fromSeconds(this.toSeconds() + 1);
63    }
64
65    public Jam_2411532007 nextNSeconds(int n) {
66        return fromSeconds(this.toSeconds() + Math.max(0, n));
67    }
68
69    public Jam_2411532007 prevSeconds() {
70        return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600));
71    }
72
73    public Jam_2411532007 prevNSeconds(int n) {
74        return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600));
75    }
76
77    public static int durasiDetik(Jam_2411532007 jaw, Jam_2411532007 jakh) {
78        return jakh.toSeconds() - jaw.toSeconds();
79    }
80
81    public String toString() {
82        return String.format("%02d:%02d:%02d", hh, mm, ss);
83    }
84
85 }

```

a. Atribut dan Constructor

Atribut hh, mm, dan ss dideklarasikan private untuk menerapkan enkapsulasi. Constructor menerima tiga parameter dan mengisi nilai tersebut ke atribut object melalui kata kunci this. Method isValid() digunakan untuk memastikan rentang jam 0-23 serta menit dan detik 0-59.

b. Operasi Konversi dan Perbandingan

Method `toSeconds()` mengubah waktu menjadi total detik sejak pukul 00:00:00. Nilai ini kemudian digunakan oleh `compareTo()` untuk membandingkan dua object dan oleh `durasiDetik()` untuk menghitung selisih. Method `equals()` dan `hashCode()` mendukung pemeriksaan kesetaraan object berdasarkan ketiga atribut waktu.

c. Operasi Waktu

Method `plus()`, `minus()`, `nextSeconds()`, `nextNSeconds()`, `prevSeconds()`, dan `prevNSeconds()` membentuk object waktu baru berdasarkan hasil operasi. Pembentukan object dilakukan melalui `fromSeconds()` sehingga hasil dapat dibatasi dalam siklus 24 jam. Pada source code saat ini, perhitungan menit di `fromSeconds()` menggunakan `total/60`. Hasil eksekusi menunjukkan nilai menit dapat melebihi 59; secara konseptual nilai menit normal dapat diperoleh dengan $(total \% 3600) / 60$. Laporan tetap menampilkan hasil program sesuai implementasi yang diuji.

2.2.2 Class JamDriver_2411532007

`JamDriver_2411532007` melakukan pengujian langsung menggunakan dua object dengan nilai tetap. Object `a` bernilai 23:59:50 dan object `b` bernilai 00:00:15. Program menampilkan kedua object, hasil penjumlahan, waktu setelah ditambah 20 detik, durasi, dan hasil perbandingan.

```

1 package pekan1_2411532007;
2
3 public class JamDriver_2411532007 {
4
5     public static void main(String[] args) {
6         Jam_2411532007 a = new Jam_2411532007(23, 59, 50);
7         Jam_2411532007 b = new Jam_2411532007(0, 0, 15);
8
9         System.out.println("a      =" + a);
10        System.out.println("b      =" + b);
11        System.out.println("a+b    =" + a.plus(b));
12        System.out.println("next 20s =" + a.nextNSeconds(20));
13        System.out.println("durasi (a,b) =" + Jam_2411532007.durasiDetik(a, b));
14        System.out.println("a compare to b =" + a.compareTo(b));
15
16    }
17
18 }
19

```

```
<terminated> JamDriver_2411532007 [Java Application] C:\Users\ndaru\.p2\pool\p
a      =23:59:50
b      =00:00:15
a+b    =00:00:05
next 20s =00:00:10
durasi (a,b) =-86375
a compare to b =1
```

Output memperlihatkan bahwa a lebih besar daripada b dalam urutan waktu pada hari yang sama, sehingga compareTo() menghasilkan 1. Durasi dihitung menggunakan b - a dan menghasilkan -86.375 detik karena waktu akhir lebih kecil daripada waktu awal tanpa penyesuaian pergantian hari. Operasi penjumlahan dan penambahan 20 detik kembali ke awal hari setelah melewati 24:00:00.

2.2.3 Class JamDriver2_2411532007

JamDriver2_2411532007 menggunakan Scanner agar nilai dua object jam dapat dimasukkan oleh pengguna. Method bantu buatJamDariInput() meminta jam, menit, dan detik secara berulang sampai nilai dinyatakan valid oleh isValid(). Setelah itu program menampilkan konversi detik, status perbandingan, durasi, waktu berikutnya, waktu sebelumnya, dan hasil penjumlahan.

```
1 package pekan1_2411532007;
2 import java.util.Scanner;
3
4 public class JamDriver2_2411532007 {
5
6     public static void main(String[] args) {
7         Scanner input = new Scanner(System.in);
8
9         System.out.println("=== Program Driver Objek Jam ===");
10
11         System.out.println("\n --- Input Jam 1 ---");
12         Jam_2411532007 j1 = buatJamDariInput(input);
13
14         System.out.println("\n --- Input Jam 2 ---");
15         Jam_2411532007 j2 = buatJamDariInput(input);
16
17         System.out.println("\n --- Hasil Operasi ---");
18         System.out.println("Jam 1 (String)      : " + j1.toString());
19         System.out.println("Jam 2 (String)      : " + j2.toString());
20         System.out.println("Jam 1 dalam detik   : " + j1.toSeconds());
21         System.out.println("Jam 2 dalam detik   : " + j2.toSeconds());
22
23         int perbandingan = j1.compareTo(j2);
24         if (perbandingan > 0) {
25             System.out.println("Status          : Jam 1 lebih lambat (setelah) jam 2");
26         } else if (perbandingan < 0) {
27             System.out.println("Status          : Jam 1 lebih awal (sebelum) jam 2");
28         } else {
29             System.out.println("Status          : Jam 1 dan jam 2 sama persis");
30         }
31
32         System.out.println("Durasi (J1 ke J2)      : " + Jam_2411532007.durasiDetik(j1, j2) + "detik");
33
34         Jam_2411532007 jNext = j1.nextSeconds();
35         System.out.println("Jam 1 Detik Berikutnya: " + jNext);
36
37         Jam_2411532007 jPrev = j1.prevSeconds();
38         System.out.println("Jam 1 Detik Berikutnya: " + jPrev);
39
40         Jam_2411532007 jHasilPlus = j1.plus(j2);
41         System.out.println("Hasil J1 + J2          : " + jHasilPlus);
42
43         input.close();
44     }
45 }
```

```

44         System.out.println("\n Program Selesai");
45     }
46
47     private static Jam_2411532007 buatJamDariInput(Scanner sc) {
48         int h, m, s;
49         while(true) {
50             System.out.print("Masukkan Jam (0-23)  :");
51             h = sc.nextInt();
52
53             System.out.print("Masukkan Menit (0-59)  :");
54             m = sc.nextInt();
55
56             System.out.print("Masukkan Detik (0-59)  :");
57             s = sc.nextInt();
58
59             if (Jam_2411532007.isValid(h, m, s))
60                 return new Jam_2411532007(h, m, s);
61             else {
62                 System.out.println("input tidak valid");
63             }
64         }
65     }
66 }
67
68 }

```

```

=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)  :10
Masukkan Menit (0-59) :20
Masukkan Detik (0-59) :30

--- Input Jam 2 ---
Masukkan Jam (0-23)  :12
Masukkan Menit (0-59) :40
Masukkan Detik (0-59) :50

--- Hasil Operasi ---
Jam 1 (String)      :10:20:30
Jam 2 (String)      :12:40:50
Jam 1 dalam detik   :37230
Jam 2 dalam detik   :45650
Status              : Jam 1 lebih awal (sebelum) jam 2
Durasi (J1 ke J2)    :8420detik
Jam 1 Detik Berikutnya:10:620:31
Jam 1 Detik Berikutnya:10:620:29
Hasil J1 + J2       :23:1381:20

Program Selesai

```

Pengujian menggunakan Jam 1 = 10:20:30 dan Jam 2 = 12:40:50. Konversi menghasilkan 37.230 detik dan 45.650 detik, sedangkan durasi keduanya sebesar 8.420 detik. `compareTo()` menyatakan Jam 1 lebih awal daripada Jam 2. Nilai menit 620 dan 1381 pada beberapa keluaran merupakan konsekuensi implementasi `fromSeconds()` yang membagi total detik langsung dengan 60 tanpa mengambil sisa terhadap satu jam.

BAB III

KESIMPULAN

Praktikum 1 berhasil menerapkan konsep Abstract Data Type melalui class `Jam_2411532007`. Data waktu direpresentasikan oleh atribut jam, menit, dan detik, sedangkan operasi terhadap data disediakan dalam bentuk method validasi, konversi, perbandingan, penjumlahan, pengurangan, perpindahan detik, dan perhitungan durasi. `JamDriver_2411532007` dan `JamDriver2_2411532007` memperlihatkan cara membuat object serta menguji method menggunakan nilai tetap maupun input pengguna.

Hasil pengujian menunjukkan bahwa sebagian besar operasi dapat dijalankan dan menghasilkan keluaran sesuai alur program. Analisis output juga menemukan bahwa formula menit pada `fromSeconds()` menyebabkan menit dapat melebihi rentang normal. Temuan tersebut menunjukkan pentingnya pengujian program tidak hanya untuk memastikan program dapat dijalankan, tetapi juga untuk memeriksa ketepatan logika setiap method.

Pada tugas praktikum, array of object digunakan untuk mengelola koleksi mobil. Operasi penambahan, pencegahan duplikasi, penampilan data, perubahan harga, dan penghapusan dapat dilakukan melalui method yang tersedia. Dengan demikian, praktikum ini memberikan dasar pemahaman mengenai class, object, enkapsulasi, ADT, dan pengelolaan kumpulan object sebelum mempelajari struktur data yang lebih kompleks.

DAFTAR PUSTAKA

Oracle. (2024). The Java Tutorials: Classes and Objects. Oracle Java Documentation.

LAPORAN TUGAS PRAKTIKUM

Tugas Praktikum 1 mengembangkan class mobil_2411532007 dan mobilDriver_2411532007. Class mobil berfungsi sebagai representasi data kendaraan sekaligus pengelola koleksi object mobil, sedangkan class driver digunakan untuk menguji seluruh operasi yang tersedia.

A. Class mobil_2411532007

Class mobil_2411532007 memiliki atribut nama, tahun, cc, harga, dan merek. Selain menyimpan data satu mobil, class ini mempunyai array koleksiMobil dengan kapasitas sepuluh elemen dan variabel jumlahMobil untuk mencatat banyaknya data yang telah tersimpan.

Kode Program Tugas 1 Class mobil_2411532007

```

1 package pekan1_2411532007;
2
3 public class mobil_2411532007 {
4     private String nama;
5     private String tahun;
6     private String cc;
7     private String harga;
8     private String merek;
9
10
11     private static final int total = 10;
12     private mobil_2411532007[] koleksiMobil = new mobil_2411532007[total];
13
14     private int jumlahMobil = 0;
15
16     public String toString() {
17         return "Mobil{" +
18             "nama=" + nama + '\'' +
19             ", tahun=" + tahun + '\'' +
20             ", cc=" + cc + '\'' +
21             ", harga=" + harga + '\'' +
22             ", merek=" + merek + '\'' +
23             '}';
24     }
25
26     public mobil_2411532007(String n, String t, String c, String h, String m) {
27         this.nama = n;
28         this.tahun = t;
29         this.cc = c;
30         this.harga = h;
31         this.merek = m;
32     }
33
34     public void infoMobil() {
35         if (jumlahMobil == 0) {
36             System.out.println("Tidak ada mobil dalam koleksi.");
37             return;
38         }
39
40         for (int i = 0; i < jumlahMobil; i++) {
41             mobil_2411532007 m = koleksiMobil[i];
42             System.out.println("mobil : " + m.getN());
43             System.out.println("tahun : " + m.getT());
44             System.out.println("cc : " + m.getC());
45             System.out.println("harga : " + m.getH());
46             System.out.println("merek : " + m.getM());
47             System.out.println("-----");
48         }
49     }
50
51     public void setN(String n) { this.nama = n; }
52     public void setT(String t) { this.tahun = t; }
53     public void setC(String c) { this.cc = c; }
54     public void setH(String h) { this.harga = h; }
55     public void setM(String m) { this.merek = m; }
56
57     public String getN() { return nama; }
58     public String getT() { return tahun; }
59     public String getC() { return cc; }
60     public String getH() { return harga; }
61     public String getM() { return merek; }
62
63     private mobil_2411532007 inKoleksi(String n) {
64         for (int i = 0; i < jumlahMobil; i++) {
65             if (koleksiMobil[i].getN().equals(n)) {
66                 return koleksiMobil[i];
67             }
68         }
69         return null;
70     }
71
72     private int getIndexKoleksi(String n) {
73         for (int i = 0; i < jumlahMobil; i++) {
74             if (koleksiMobil[i].getN().equals(n)) {
75                 return i;
76             }
77         }
78         return -1;
79     }
80 }
81
82
83
84
85

```

```

85
86● public void tambahMobil(String n, String t, String c, String h, String m) {
87
88    if (inKoleksi(n) != null) {
89        System.out.println("mobil " + n + " ini sudah ada di dalam koleksi");
90        return;
91    }
92
93
94    if (jumlahMobil >= total) {
95        System.out.println("Koleksi mobil sudah penuh (Maksimal " + total + ")");
96        return;
97    }
98
99    mobil_2411532007 mobilbaru = new mobil_2411532007(n, t, c, h, m);
100    koleksiMobil[jumlahMobil] = mobilbaru;
101    jumlahMobil++;
102
103    System.out.println("mobil " + mobilbaru.getN() + " ditambahkan ke dalam koleksi");
104 }
105
106● public void hapusMobil(String n) {
107    if (jumlahMobil == 0) {
108        System.out.print("anda tidak memiliki mobil");
109        return;
110    }
111
112    int index = getIndexKoleksi(n);
113
114    if (index != -1) {
115        mobil_2411532007 m = koleksiMobil[index];
116
117        for (int i = index; i < jumlahMobil - 1; i++) {
118            koleksiMobil[i] = koleksiMobil[i + 1];
119        }
120
121        koleksiMobil[jumlahMobil - 1] = null;
122        jumlahMobil--;
123
124        System.out.println(" Mobil yang berhasil dihapus" + m);
125    } else {
126        System.out.print("tidak ada mobil yang dihapus");
127    }
128 }
129
130
131● public void hapusMobil(String n) {
132    if (jumlahMobil == 0) {
133        System.out.print("anda tidak memiliki mobil");
134        return;
135    }
136
137    int index = getIndexKoleksi(n);
138
139    if (index != -1) {
140        mobil_2411532007 m = koleksiMobil[index];
141
142        for (int i = index; i < jumlahMobil - 1; i++) {
143            koleksiMobil[i] = koleksiMobil[i + 1];
144        }
145
146        koleksiMobil[jumlahMobil - 1] = null;
147        jumlahMobil--;
148
149        System.out.println(" Mobil yang berhasil dihapus" + m);
150    } else {
151        System.out.println("tidak ada mobil yang dihapus");
152    }
153 }
154
155
156● public void ubahHargaMobil(String n, String h) {
157    if (jumlahMobil == 0) {
158        System.out.println("tidak ada mobil yang ada untuk diubah");
159        return;
160    }
161
162    mobil_2411532007 m = inKoleksi(n);
163
164    if (m != null) {
165        m.setH(h);
166        System.out.println("Harga mobil " + n + " berhasil diubah menjadi " + h);
167    } else {
168        System.out.print("tidak ada mobil yang anda cari ");
169    }
170 }
171
172 }
173
174 }

```

1. Struktur Data dan Enkapsulasi

Seluruh atribut utama dideklarasikan private. Data dapat dibaca melalui getter getN(), getT(), getC(), getH(), dan getM(), serta diubah melalui setter yang sesuai. Array koleksiMobil menyimpan referensi object mobil dan dibatasi oleh konstanta total sebesar 10.

2. Pencarian Data

Method `inKoleksi()` menelusuri array untuk mengembalikan object dengan nama yang dicari. Method `getIndexKoleksi()` melakukan proses serupa, tetapi mengembalikan posisi indeks. Kedua method tersebut bersifat private karena hanya digunakan sebagai operasi pendukung di dalam class.

3. Penambahan, Perubahan, dan Penghapusan

Method `tambahMobil()` menolak nama yang sudah ada dan memastikan kapasitas array belum penuh sebelum membuat object baru. Method `ubahHargaMobil()` mencari object berdasarkan nama kemudian memperbarui atribut harga. Method `hapusMobil()` mencari indeks data, menggeser seluruh elemen setelah indeks tersebut satu posisi ke kiri, mengosongkan elemen terakhir, dan mengurangi jumlahMobil.

B. Class `mobilDriver_2411532007`

Class `mobilDriver_2411532007` menjadi skenario pengujian tugas. Program menambahkan tiga mobil, mencoba memasukkan data duplikat, menampilkan koleksi, mengubah harga, mencoba mengubah data yang tidak tersedia, menghapus mobil, mencoba menghapus data yang tidak tersedia, dan menampilkan koleksi akhir.

```

1 package pekan1_2411532007;
2
3 public class mobilDriver_2411532007 {
4
5     public static void main(String[] args) {
6         mobil_2411532007 dealer = new mobil_2411532007("punya admin hahaha", "2024", "0", "0", "Admin");
7
8         System.out.println("=== 1. Menambah Mobil ===");
9         dealer.tambahMobil("Avanza", "2020", "1300cc", "200 Juta", "Toyota");
10        dealer.tambahMobil("Brio", "2021", "1200cc", "180 Juta", "Honda");
11        dealer.tambahMobil("Pajero", "2019", "2400cc", "500 Juta", "Mitsubishi");
12
13        System.out.println("\n=== 2. Coba Tambah Mobil Duplikat ===");
14        dealer.tambahMobil("Avanza", "2022", "1500cc", "250 Juta", "Toyota");
15
16        System.out.println("\n=== 3. Lihat Semua Koleksi (Info) ===");
17        dealer.infoMobil();
18
19        System.out.println("\n=== 4. Ubah Harga Mobil ===");
20        dealer.ubahHargaMobil("Brio", "175 Juta");
21        dealer.ubahHargaMobil("Civic", "300 Juta");
22
23        System.out.println("\n=== 5. Lihat Koleksi Setelah Ubah Harga ===");
24        dealer.infoMobil();
25
26        System.out.println("\n=== 6. Hapus Mobil ===");
27        dealer.hapusMobil("Pajero");
28        dealer.hapusMobil("Ferrari");
29
30        System.out.println("\n=== 7. Lihat Koleksi Akhir ===");
31        dealer.infoMobil();
32    }
33 }
34 }
35

```

C. Hasil dan Analisis Tugas

```

=== 1. Menambah Mobil ===
mobil Avanza ditambahkan ke dalam koleksi
mobil Brio ditambahkan ke dalam koleksi
mobil Pajero ditambahkan ke dalam koleksi

=== 2. Coba Tambah Mobil Duplikat ===
mobil Avanza ini sudah ada di dalam koleksi

=== 3. Lihat Semua Koleksi (Info) ===
mobil :Avanza
tahun :2020
cc :1300cc
harga :200 Juta
merek :Toyota
-----
mobil :Brio
tahun :2021
cc :1200cc
harga :180 Juta
merek :Honda
-----
mobil :Pajero
tahun :2019
cc :2400cc
harga :500 Juta
merek :Mitsubishi
-----

=== 4. Ubah Harga Mobil ===
Harga mobil Brio berhasil diubah menjadi 175 Juta
tidak ada mobil yang anda cari
=== 5. Lihat Koleksi Setelah Ubah Harga ===
mobil :Avanza
tahun :2020
cc :1300cc
harga :200 Juta
merek :Toyota
-----
mobil :Brio
tahun :2021
cc :1200cc
-----
mobil :Pajero
tahun :2019
cc :2400cc
harga :500 Juta
merek :Mitsubishi
-----

=== 6. Hapus Mobil ===
Mobil yang berhasil dihapusMobil{nama='Pajero', tahun='2019', cc='2400cc', harga='500 Juta', merek='Mitsubishi'}
tidak ada mobil yang dihapus

=== 7. Lihat Koleksi Akhir ===
mobil :Avanza
tahun :2020
cc :1300cc
harga :200 Juta
merek :Toyota
-----
mobil :Brio
tahun :2021
cc :1200cc
harga :175 Juta
merek :Honda
-----

```

Pada tahap penambahan, Avanza, Brio, dan Pajero berhasil dimasukkan ke array. Upaya menambahkan Avanza kedua kali ditolak karena method `inKoleksi()` menemukan nama yang sama. Hal ini menunjukkan bahwa mekanisme pencegahan duplikasi telah bekerja berdasarkan atribut nama.

Perubahan harga Brio menjadi 175 Juta berhasil dilakukan, sedangkan pencarian Civic menghasilkan pesan bahwa mobil tidak ditemukan. Penghapusan Pajero berhasil dan elemen setelah posisi penghapusan digeser agar tidak meninggalkan celah. Upaya menghapus Ferrari juga menghasilkan pesan bahwa tidak ada mobil yang dihapus. Koleksi akhir hanya berisi Avanza dan Brio dengan harga Brio yang telah diperbarui.

Secara keseluruhan, tugas praktikum telah menerapkan array of object dan operasi dasar pengelolaan data. Pencarian masih menggunakan pencocokan nama yang bersifat case-sensitive dan kompleksitas linear $O(n)$. Untuk kapasitas maksimal sepuluh object, pendekatan tersebut masih memadai serta mudah dipahami sebagai pengantar sebelum menggunakan struktur data dinamis.